

```
In [1]: import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
In [3]: # Load the dataset
file_path = "C:\\Users\\lavan\\Desktop\\Machine Learning for Cybersecurity_ Lab 5-M
try:
    logfiles = pd.read_csv(file_path)
except Exception as e:
    print(f"Error loading dataset: {e}")
    exit()
```

```
In [5]: # Inspecting the dataset
print("Dataset Columns:")
print(logfiles.columns)
```

```
Dataset Columns:
Index(['class_name', '00', '000', '0', '46_1', '20424', '000214a0',
      '00a0c91efb8b', '00aa003e56f8', '00c04fc295ee',
      ...,
      'wmiprvse', 'wpdbusenumroot', 'write', 'writefile', 'wudfhost', 'x86',
      'x86_microsoft', 'yax', 'zip', 'zones'],
      dtype='object', length=1001)
```

```
In [7]: # Handle potential -1 values in the target column
target_column = 'class_name' # Replace this with your actual target column name
if target_column not in logfiles.columns:
    raise KeyError(f"'{target_column}' not found in the dataset columns.")
```

```
In [9]: # Convert -1 values to a valid label (for example, 0)
logfiles[target_column] = logfiles[target_column].replace(-1, 0)
```

```
In [11]: # Prepare the data
X = logfiles.drop(columns=[target_column]).values
y = logfiles[target_column].values
```

```
In [13]: # Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

```
In [15]: # Standardizing the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [17]: # Convert to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train)
y_train_tensor = torch.LongTensor(y_train)
```

```
X_test_tensor = torch.FloatTensor(X_test)
y_test_tensor = torch.LongTensor(y_test)
```

```
In [19]: # Define the neural network model
class SimpleNN(nn.Module):
    def __init__(self, input_size):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(input_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 2) # Assuming binary classification; adjust for m

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
In [23]: # Instantiate the model
input_size = X_train.shape[1] # Number of features
model = SimpleNN(input_size)

# Define Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
In [25]: # Training the model
num_epochs = 100
train_losses = []

for epoch in range(num_epochs):
    model.train()
    optimizer.zero_grad() # Zero the parameter gradients

    outputs = model(X_train_tensor) # Forward pass
    loss = criterion(outputs, y_train_tensor) # Compute Loss
    loss.backward() # Backward pass
    optimizer.step() # Optimize the weights

    train_losses.append(loss.item()) # Store the Loss value

# Print the Loss every 10 epochs
if (epoch + 1) % 10 == 0:
    print(f'Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}')
```

```
Epoch [10/100], Loss: 0.0577
Epoch [20/100], Loss: 0.0083
Epoch [30/100], Loss: 0.0008
Epoch [40/100], Loss: 0.0001
Epoch [50/100], Loss: 0.0001
Epoch [60/100], Loss: 0.0000
Epoch [70/100], Loss: 0.0000
Epoch [80/100], Loss: 0.0000
Epoch [90/100], Loss: 0.0000
Epoch [100/100], Loss: 0.0000
```

```
In [27]: # Testing the model
model.eval()
with torch.no_grad():
    test_outputs = model(X_test_tensor)
    _, predicted = torch.max(test_outputs.data, 1)
```

```
In [29]: # Performance metrics
accuracy = accuracy_score(y_test, predicted.numpy())
f1 = f1_score(y_test, predicted.numpy(), average='weighted')
precision = precision_score(y_test, predicted.numpy(), average='weighted')
recall = recall_score(y_test, predicted.numpy(), average='weighted')
conf_matrix = confusion_matrix(y_test, predicted.numpy())

print("\nPerformance Metrics:")
print(f'Accuracy: {accuracy:.4f}')
print(f'F1 Score: {f1:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print("Confusion Matrix:")
print(conf_matrix)
```

Performance Metrics:

Accuracy: 0.9310

F1 Score: 0.9310

Precision: 0.9310

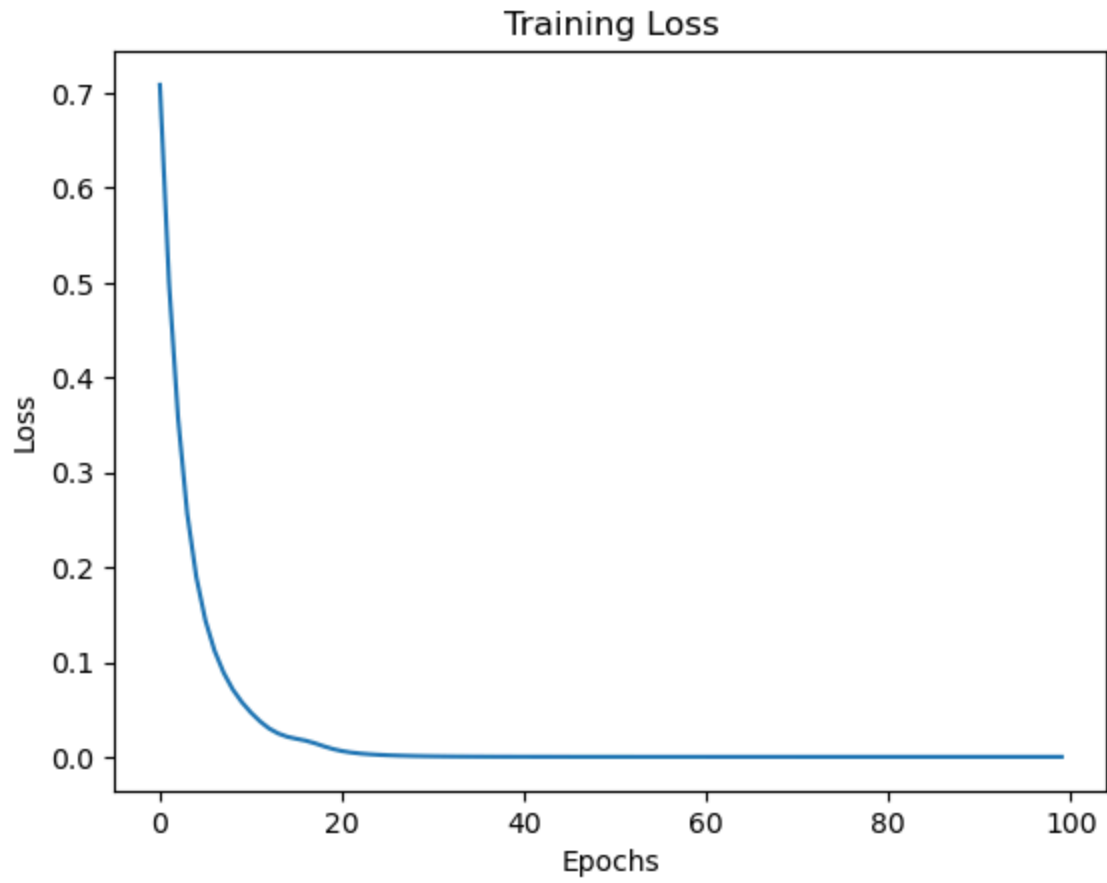
Recall: 0.9310

Confusion Matrix:

```
[[19  1]
 [ 1  8]]
```

```
In [31]: # Plot training loss
import matplotlib.pyplot as plt

plt.plot(train_losses)
plt.title('Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.show()
```



In []: